POWERPOINT PREPARATION GUIDE

Delayed Gradients in CPU-Based Neural Network Training

A slide-by-slide handoff for the teammate preparing the project presentation

CENTRAL QUESTION

Can boundedly stale gradients reduce synchronization waiting while the model still learns well?

Recommended deck

Prepare about 9 slides for a short class presentation. Adjust the slide count and depth to the professor's time limit. The presentation should explain the operating-system idea, show the experiment controls, and interpret the measured speed-versus-learning tradeoff.

- Lead with the research question and the observed result, not a tour of every source file.
- Use the supplied charts from the project workbook. Do not redraw values by hand.
- State that the experiment uses a small MNIST subset and that asynchronous runs vary.
- Treat the measurements as evidence from this implementation and machine, not a universal speedup claim.

What this brief is based on

Project settings and results supplied by the implementation team: a 784-32-10 MLP, MNIST, two worker threads, 5,000 training examples per epoch, 1,000 test examples, learning rate 0.01, and the recorded tau_max experiments.

Slide-by-slide story: slides 1-5

Slide	Key content	What to explain
1. Title and question	Delayed Gradients in CPU-Based Neural Network Training	Ask whether bounded stale gradients can reduce waiting without harming useful learning. Name the team and course.
2. Why synchronization matters	Workers depend on activations and gradients from other workers.	Use a simple two-worker diagram. Show that a worker can wait for input; more threads alone do not guarantee useful parallel work.
3. The model and data	MNIST; 784 inputs -> 32 ReLU neurons -> 10 digit outputs.	Mention normalized pixels, cross-entropy, 5,000 training examples per epoch, and 1,000 test examples. The full MNIST split has 60,000 train and 10,000 test images.
4. What gradient age means	age = current hidden-layer version - gradient origin version	Explain that the gradient is real and computed by backpropagation, but may refer to an older hidden-layer weight version. tau_max is the age limit.
5. Three implementations	Sequential; synchronous workers (1 in flight); delayed workers (up to 8 in flight).	Show the comparison is controlled around the same architecture, data subset, seed, and learning rate. Delayed runs sweep tau_max = 0, 1, 2, 4, 8, 16.

Suggested visual for slide 4

Draw two owner-workers. Worker 1 owns the output layer and sends a gradient with the batch ID and origin version. Worker 0 compares that origin version with its current version, then accepts or discards the gradient according to tau_max.

Do not say

Do not describe the gradient as fabricated or predicted. It is a genuine gradient from an earlier model state. Avoid claiming that `tau_max=0, max_in_flight=8` is the same execution as the synchronous reference; those settings differ in queue concurrency.

Slide-by-slide story: slides 6-9

Slide	Key content	What to explain
6. Experimental method	5 epochs; 0.01 learning rate; seed 12345; fixed selected subsets.	Separate training time from evaluation time. Report test accuracy and loss at epoch checkpoints. Explain that time-to-90% is only measured at those checkpoints.
7. Learning curves	Test accuracy vs epoch and test loss vs epoch.	Use the two charts in epoch_curves.xlsx. Discuss learning trend and any test-loss/accuracy fluctuations. Label runs clearly; tau_max=4 has repeated runs.
8. Delay limit tradeoff	Time to 90% accuracy vs tau_max; accepted-gradient rate and final accuracy.	Use the time-to-target chart and epoch_comparison.csv. tau_max=4 reached 90% at epoch 2 in all three recorded five-epoch runs (7.47, 8.02, and 8.52 seconds). tau_max=0 with 8 in flight did not reach 90%.
9. Worker timing and conclusion	Synchronous and delayed worker wait measurements; limitations.	Explain input-queue waiting and the observed imbalance. Conclude that the measured tau_max=4 run was faster than the synchronous run, while final accuracy was close. Results are limited and variable.

Important chart notes

- Learning curves: x-axis is epoch; use test accuracy (%) and test loss as separate charts.
- Time-to-target chart: x-axis is tau_max; y-axis is seconds to first checkpoint at or above 90%. tau_max=0 is omitted because the target was not reached.
- Do not call completed-example rate useful-gradient throughput: rejected hidden-layer gradients still count as completed examples. Pair throughput with accepted updates and acceptance percentage.
- Per-worker wait totals can overlap and can exceed wall-clock time. They are queue-pop wait measurements, not direct CPU utilization.

Results to use carefully

The table below summarizes the primary five-epoch runs saved in epoch_comparison.csv. A later instrumented run added a third tau_max=4 observation; keep that distinction visible rather than silently overwriting earlier measurements.

Configuration	Final test accuracy	Training time	First >=90%
Sequential run 1	91.4%	26.92 s	10.88 s
Sequential run 2	91.4%	29.63 s	11.38 s
Synchronous (0, 1 in flight)	91.4%	28.61-28.95 s	11.54-11.63 s
Delayed tau=0 (8 in flight)	88.9%	10.00 s	Not reached
Delayed tau=1 (8 in flight)	91.1%	11.18 s	11.18 s
Delayed tau=2 (8 in flight)	90.5%	15.76 s	12.71 s
Delayed tau=4 (8 in flight)	91.0-92.1%	18.73-21.14 s	7.47-8.52 s
Delayed tau=8 (8 in flight)	92.2%	26.58 s	10.76 s

Worker timing example

Run	W0 active path	W0 input wait	W1 active path	W1 input wait
Synchronous (0,1)	27.146 s	2.851 s	0.586 s	29.421 s
Delayed (4,8)	18.690 s	1.102 s	0.817 s	18.966 s

These are individual runs, not a statistical proof. A safe conclusion is: in the measured runs, bounded delay was associated with lower elapsed time and lower Worker 1 input-queue waiting, with comparable but not identical final accuracy. More repetitions would strengthen the claim.

Timing definition: worker active-path time includes processing and queue pushes; input-queue wait is time inside blocking queue pops. The metrics do not directly measure CPU utilization or queue depth.

Design and speaking guidance

Keep the deck visual and make each slide answer one question. Use a consistent color for each mode: sequential, synchronous, delayed. Use the same color mapping in all charts.

A clear 60-second conclusion

> **“The project implements sequential, synchronous, and bounded delayed-gradient training on CPU. In the measured five-epoch runs, a delay bound of 4 reached 90% test accuracy earlier than the synchronous configuration, while the strictest asynchronous setting discarded many hidden-layer gradients and failed to reach 90%. The results show a speed-quality tradeoff in this implementation; they do not establish a universal speedup.”**

Avoid these overclaims

- Do not claim a general CPU speedup from one machine and a small subset.
- Do not claim tau_max=4 always improves accuracy; three runs varied from 91.0% to 92.1%.
- Do not equate completed-example throughput with accepted hidden-layer updates.
- Do not describe queue-pop wait as total worker idle time or CPU utilization.
- Do not imply the team measured memory, queue depth, or CPU utilization unless additional measurements are collected.

Likely questions

Question	Short answer direction
What is stale?	The gradient was computed using an earlier hidden-layer version; age is a version difference.
Why discard gradients?	The policy rejects gradients older than tau_max to bound staleness; this trades updates for less waiting.
Why can accuracy change between runs?	Asynchronous worker scheduling changes update order; a few repetitions show variability.
What is the main OS contribution?	Fixed owner threads, bounded producer-consumer queues, condition-variable blocking, version metadata, and measured waiting behavior.

Files to share with the teammate

Share the items in the first section so the teammate can build the deck. Share source and raw experiment files if they need to verify details or reproduce a run.

Priority	File or folder	Why it helps
Essential	PowerPoint_Preparation_Guide_Delayed_Gradients.pdf	This slide plan, results summary, and claim guidance.
Essential	README.md	Project goal, model, build/run commands, and reported results.
Essential	results/epoch_comparison.csv	Summary comparison across configurations, including time-to-target.
Essential	results/epoch_curves.xlsx	Workbook containing the learning-curve and time-to-target charts.
Essential	results/epoch_curves.csv	Underlying per-epoch data for accuracy and loss plots.
Helpful	results/demo_script.md	Concise explanation and suggested demo sequence.
Helpful	results/worker_timing_runs.txt (if saved)	Raw synchronous and delayed worker wait outputs.
Reference	Approach_1_Delayed_Gradients_Implementation_Guide.pdf	Original project guidance/rubric context.
Reference	src/ folder	Implementation details for questions about queues, ownership, versions, and updates.
Optional	results/ raw logs and experiment scripts	Traceability for repeated sweeps and benchmark runs.

Usually do not send

- Do not send the full MNIST data folder just to make slides; it is not needed to read the charts. Send it only if the teammate must rerun training.
- Do not rely on the .exe files as portable evidence. If reproduction is needed, share source, build commands, and the data instructions instead.
- Avoid sending a large collection of unrelated logs when the summary CSV and the relevant raw logs answer the slide questions.

Before handing off

- Confirm the charts open in epoch_curves.xlsx and their axes/legends are readable.
- Tell the teammate which three configurations to demonstrate: sequential, synchronous, and delayed tau_max=4 with 8 batches in flight.
- Ask them to keep the caveats from this guide on the results/conclusion slide.